Fine-Tuning

Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Agenda

In this session, we'll discuss:

- Benefits of Fine-Tuning and LLM
- Parameter-Efficient Fine-Tuning and its Methods
- Process of Fine-Tuning an LLM using Parameter-Efficient Fine-Tuning (PEFT) and Low-Rank Adaptation (LoRA)

Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why Fine-Tuning?

Fine-tuning is the process of adjusting the weights of pre-trained models to improve performance on a specific task or domain. It is the last resort if the behavior of the model cannot be controlled through careful prompt engineering and retrieval augmentation.

Benefits

- LLMs adapt to task-specific or domain-specific instructions.
- LLMs follow instructions on the format (e.g., JSON) or quality of the response (e.g., formal tone) expected.

Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Adapting Pre-trained LLMs

Mistral

```
MistralForCausalLM(
  (model): MistralModel(
    (embed_tokens): Embedding(32000, 4096)
    (layers): ModuleList(
      (0-31): 32 x MistralDecoderLayer(
        (self_attn): MistralAttention(
          (q_proj): Linear4bit(in_features=4096, out_features=4096, bias=False)
          (k_proj): Linear4bit(in_features=4096, out_features=1024, bias=False)
          (v_proj): Linear4bit(in_features=4096, out_features=1024, bias=False)
          (o_proj): Linear4bit(in_features=4096, out_features=4096, bias=False)
          (rotary_emb): LlamaRotaryEmbedding()
        )
        (mlp): MistralMLP(
          (gate_proj): Linear4bit(in_features=4096, out_features=14336, bias=False)
          (up_proj): Linear4bit(in_features=4096, out_features=14336, bias=False)
          (down_proj): Linear4bit(in_features=14336, out_features=4096, bias=False)
          (act_fn): SiLU()
        )
        (input_layernorm): MistralRMSNorm()
        (post_attention_layernorm): MistralRMSNorm()
      )
    )
    (norm): MistralRMSNorm()
  )
  (lm_head): Linear(in_features=4096, out_features=32000, bias=False)
)
```

Methods

Fine-tune all parameters — Compute & data intensive

Transfer Learning — Risk of catastrophic forgetting

Parameter Efficient Fine Tuning

Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Parameter-Efficient Fine-Tuning (PEFT)

PEFT Methods

Only small number of parameters are adjusted

LoRA, QLoRA

Low-rank decomposition of weights is updated

Prefix Tuning

Trainable task-specific prefix vectors are added to Transformer layers

Prompt Tuning

Prompt-specific tokens are added to the input for conditional generation

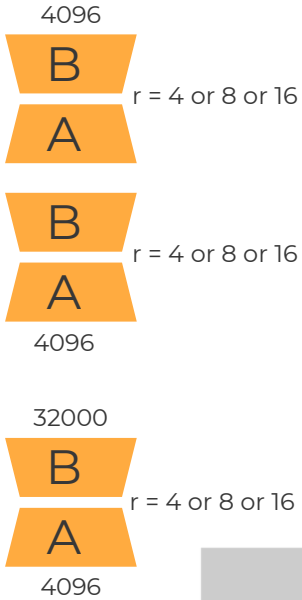
Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Low-Rank Adaptation (LoRA)

Frozen Base + **Adapters** = **Final Model**

```
MistralForCausalLM(
  (model): MistralModel(
    (embed_tokens): Embedding(32000, 4096)
    (layers): ModuleList(
      (0-31): 32 x MistralDecoderLayer(
        (self_attn): MistralAttention(
          (q_proj): Linear4bit(in_features=4096, out_features=4096, bias=False)
          (k_proj): Linear4bit(in_features=4096, out_features=1024, bias=False)
          (v_proj): Linear4bit(in_features=4096, out_features=1024, bias=False)
          (o_proj): Linear4bit(in_features=4096, out_features=4096, bias=False)
          (rotary_emb): LlamaRotaryEmbedding()
        )
        (mlp): MistralMLP(
          (gate_proj): Linear4bit(in_features=4096, out_features=14336, bias=False)
          (up_proj): Linear4bit(in_features=4096, out_features=14336, bias=False)
          (down_proj): Linear4bit(in_features=14336, out_features=4096, bias=False)
          (act_fn): SiLU()
        )
        (input_layernorm): MistralRMSNorm()
        (post_attention_layernorm): MistralRMSNorm()
      )
    )
    (norm): MistralRMSNorm()
  )
  (lm_head): Linear(in_features=4096, out_features=32000, bias=False)
)
```



Low-rank adapters (the product of matrices A & B) are attached to specific layers of the frozen base model. # of parameters in these adapters are < 10% of the frozen base and only these parameters are trained.

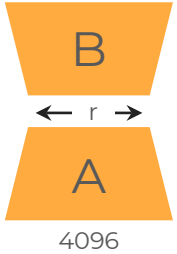
Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.
This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Low-Rank Adaptation (LoRA)

Frozen Base

```
MistralForCausalLM(
  (model): MistralModel(
    (embed_tokens): Embedding(32000, 4096)
    (layers): ModuleList(
      (0-31): 32 x MistralDecoderLayer(
        (self_attn): MistralAttention(
          (q_proj): Linear4bit(in_features=4096, out_features=4096, bias=False)
          (k_proj): Linear4bit(in_features=4096, out_features=1024, bias=False)
          (v_proj): Linear4bit(in_features=4096, out_features=1024, bias=False)
          (o_proj): Linear4bit(in_features=4096, out_features=4096, bias=False)
          (rotary_emb): LlamaRotaryEmbedding()
        )
        (mlp): MistralMLP(
          (gate_proj): Linear4bit(in_features=4096, out_features=14336, bias=False)
          (up_proj): Linear4bit(in_features=4096, out_features=14336, bias=False)
          (down_proj): Linear4bit(in_features=14336, out_features=4096, bias=False)
          (act_fn): SiLU()
        )
        (input_layernorm): MistralRMSNorm()
        (post_attention_layernorm): MistralRMSNorm()
      )
    )
    (norm): MistralRMSNorm()
  )
  (lm_head): Linear(in_features=4096, out_features=32000, bias=False)
)
```

Adapters



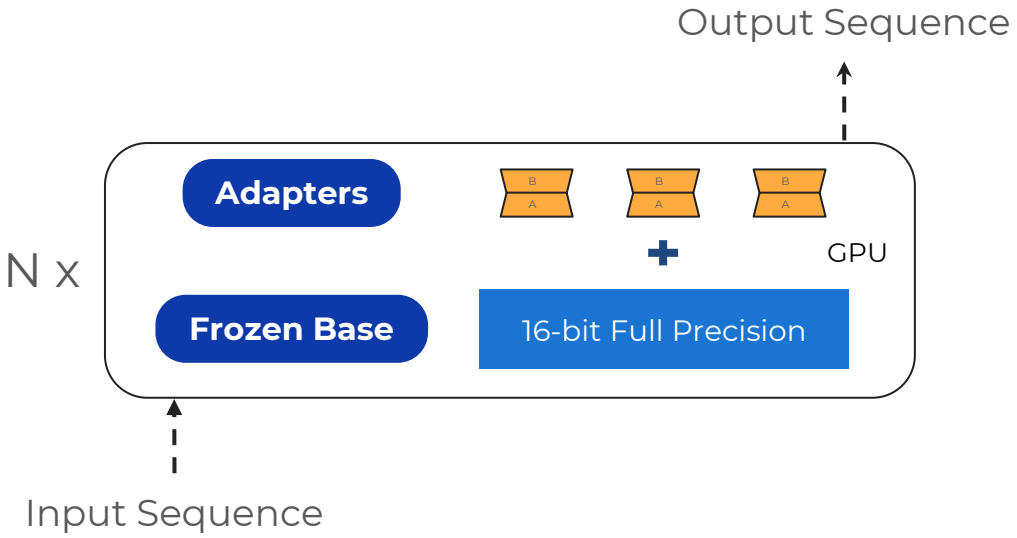
Parameters:
Rank (r)
Dimension of A & B

Scaling Coefficient (α)
Scaling factor for A & B

α scales the values of the adapter matrices. Typically, the value of α is set to be 16 while r is varied as α, $\alpha/2$, and $\alpha/4$. Number of adapters and the best r are picked through hyperparameter tuning.

Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.
This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Low-Rank Adaptation (LoRA)

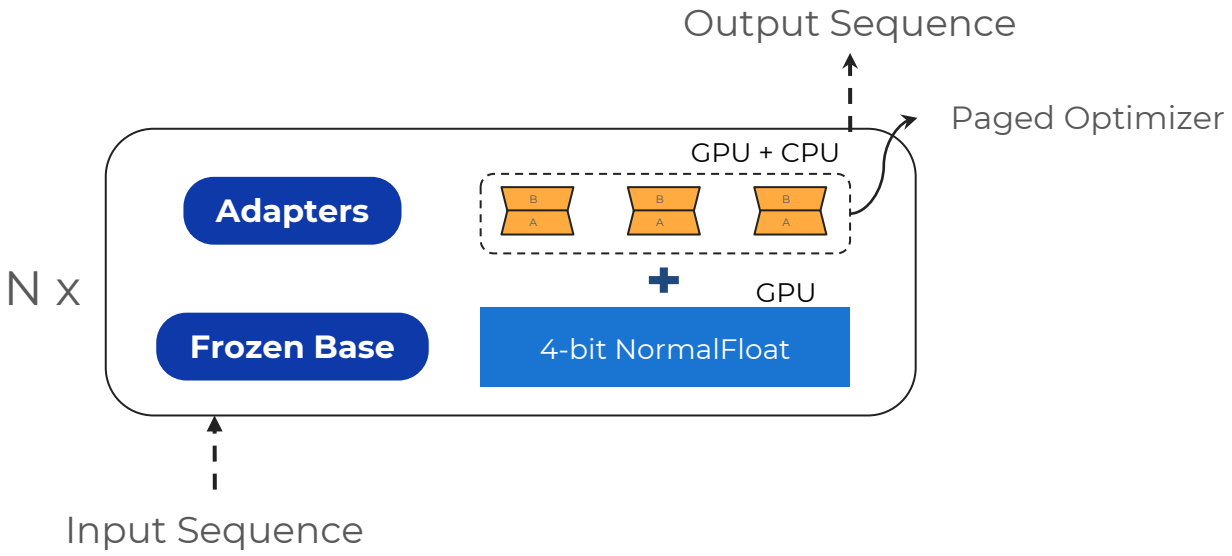


In LoRA, we inject adapter layers to specific components of the frozen full precision base model. Training is limited to these adapters only. The full precision model and the adapters are hosted on GPU(s) during training.

Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Quantized Low-Rank Adaptation (QLoRA)

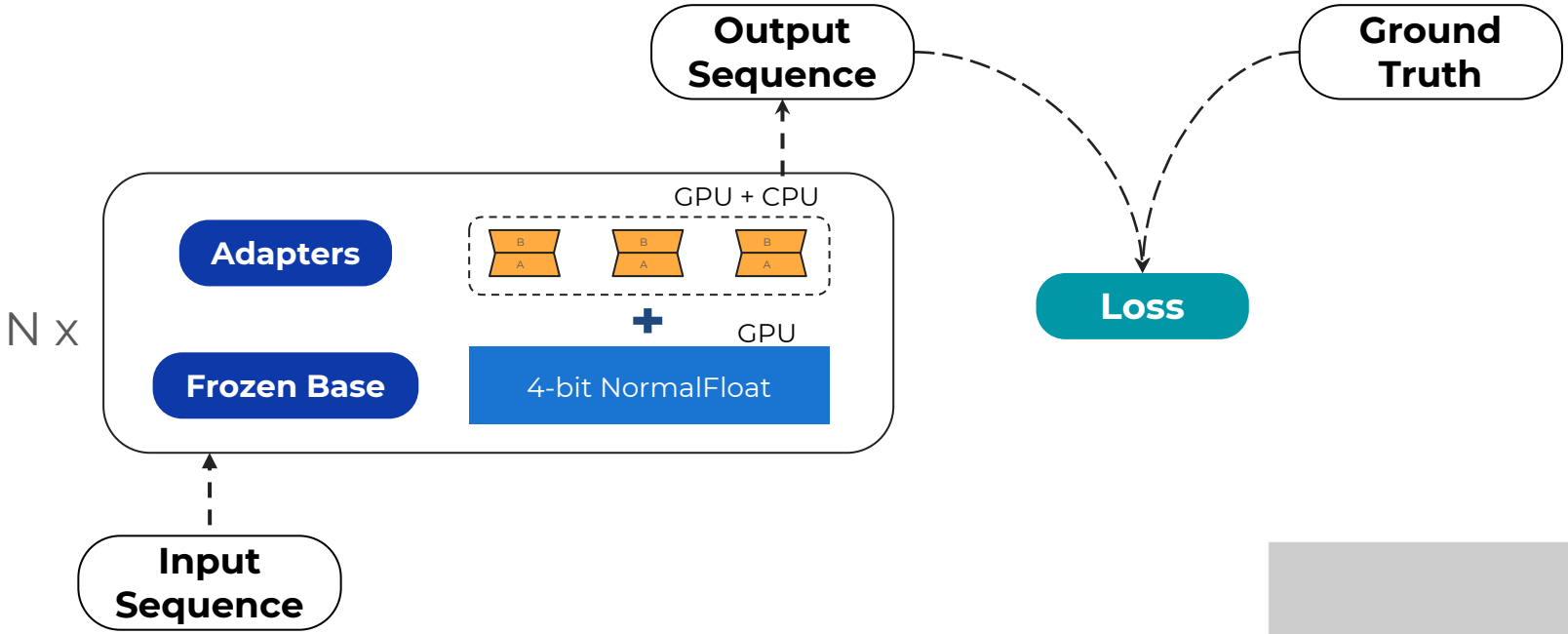


In QLoRA, the frozen base is 4-bit quantized. Further a paged optimizer is used to efficiently shuffle the adapter parameters between GPU and CPU during training. This approach reduces the GPU footprint tremendously

Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Quantized Low-Rank Adaptation (QLoRA)



Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Summary

Here is a quick recap:

- Fine-Tuning is useful when the quality or format of LLM responses is not up to the mark.
- Parameter-Efficient Fine-Tuning (PEFT) methods focus on reducing the number of training parameters without loss in performance.
- Low-Rank Adaptation (LoRA) is an effective PEFT method where low-rank matrix decompositions are trained instead of all the parameters.
- Quantized Low-Rank Adaptation (QLoRA) reduces the computational complexity of LoRA by quantizing base model parameters and paged optimization.

Proprietary content. ©Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by samyukthakcontactinfo@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.